Transforming Relational Data into Graph Machine Learning Applications: Common Approaches and Examples

Rajiv Sambasivan

October 25, 2025

1 Overview and Scope

Graphs are a common analysis tool in data science and machine learning. In some cases, the need to use graphs is recognized upfront. The data model is designed to be a graph from the start. In other cases, the need to use graphs is recognized during the analysis phase. In such cases, the data is typically in a relational format. For example, data may come from an ERP (Enterprise Resource Planning)system, a CRM (Customer Relationship Management)system or Sales system (Salesforce). The challenge is to convert the relational data into a graph structure that is suitable for analysis. This document provides an overview of the concepts related to this transformation. The scope of the document is limited to the transformation of relational data into graph structures. Some familiarity with the relational model and the entity-relationship model is assumed. On the target side, the scope of this document is limited to the construction of homogeneous, bipartite and heterogeneous graphs. The document does not cover advanced topics such as hypergraphs, temporal graphs etc. The document stops at the point of graph construction. The document does not cover graph analysis or graph learning.

2 Motivations to Use Graphs

Typically when analysts want to explore graphs as a data representation, *heterogeneity* in the data is a key motivation. Heterogeneity is a term that originates in statistics. In statistics, heterogeneity refers to the presence of sub-populations within an overall population that exhibit different characteristics. For example, in a clinical trial, patients may respond differently to a treatment based on factors such as age. A company may have different customer segments that exhibit different purchasing behaviors. Heterogeneity is a common characteristic of business data. Graphs provide a natural way to represent and analyze

heterogeneous data. Graphs can capture the relationships between different entities in the data. Graphs can also capture the different types of entities in the data. This makes graphs a powerful tool for analyzing heterogeneous data.

How hetrogeneity manifests itself in data depends on the type of analysis being performed. There are three common types of data analysis: *cross-sectional, longitudinal* and *panel*. Each type of analysis has its own characteristics and challenges when it comes to identifying and accounting for heterogeneity in the data.

In a *cross-sectional* analysis, sources of heterogeneity may be identified through domain expertise or by applying data-driven techniques. A dataset where you are modeling housing prices over a wide geographic region at a specific point in time is an example of a *cross-sectional* dataset. A real estate domain expert may have developed ways to partition the housing market into segments where each segment has similar relationship between the house price and the characteristics of the hous. Alternatively, you can use machine learning to systematically uncover these sources. For example, you can use a Classification and Regression Tree (CART)[1] to identifiy regions that have a similar predictor-response characteristics and then develop models for each of these segments [15]. The regions identified by the regression tree are the sources of heterogeneity in the data.

In a *longitudinal analysis*, heterogeneity occurs over time. A study where you are monitoring the blood glucose of a subject over time, or, a study where you are tracking the price of a commodity like coffee over time is an example of a *longitudinal analysis*. Tracking changes with respect to time using *change point analysis* can tell you when and how many changes have occurred in the period that you are monitoring. See [14] for an example. Change point detection is a canonical time series analysis task and many methods such as the *Matrix Profile*[18] have been used to identify change points. Each segment between change points can be treated as a homogeneous segment. You can then model each segment separately. The segments identified by change point analysis are the sources of heterogeneity in the data.

In *Panel Data*, you need to account for sources of variation you have observed with both *cross-sectional* and *longitudinal* data. A company may track the purchasing behavior of a set of customers over time. In such cases, you need to account for heterogeneity across customers as well as heterogeneity over time.

3 Guidelines for Implementation

This section provides guidelines for implementing the transformation of relational data into graph structures. Definitions of key terms used in this section are provided in the Appendix.

The implementation consists of an initial assessment of the data, analysis for heterogeneity and a graph mapping. The guidelines for performing these steps are given below.

3.1 Initial Assessment

The initial assessment is done on the raw data obtained from the source systems. For the initial assessment, use a `jupyter notebook` to do the following:

1. Data Sourcing: Work with the data engineering team to understand how to get extracts from the source relational systems. Understand the data extraction process and the frequency of data extraction. Understand the data formats and the schema of the extracts. The data that you will source will be an estimate based on human judgement and understanding of what your use case aims to accomplish. This will be refined in subsequent analysis. Document the rationale for the estimate and the assumptions in a knowledge management system.

2. Data Profiling: Perform data profiling on the extracts to understand the data quality. Identify missing values, outliers, and inconsistencies in the data. Document your findings in a knowledge management system.

3. Data Selection: Based on your understanding of the use case, use an appropriate set of techniques, for example, feature selection techniques, to select the relevant data for your use case. Document your rationale for data selection in a knowledge management system.

4. Data Cleaning: Perform data cleaning to address the issues identified in the data profiling step. This may include handling missing values, removing outliers, and correcting inconsistencies in the data. Document your data cleaning process in a knowledge management system.

5. Data Transformation: Perform data transformation to convert the data into a format suitable for analysis. This may include normalization, aggregation, and encoding categorical variables. Document your data transformation process in a knowledge management system.

6. Data Exploration: Perform exploratory data analysis to understand the characteristics of the data. This may include visualizations, summary statistics, and correlation analysis. The purpose of this step is to develop an initial understanding of the data and formulate a modeling plan. Document your findings in a knowledge management system.

7. Modeling Plan: Develop a preliminary modeling plan based on your understanding of the use case and the data. Document your modeling plan in a knowledge management system. The purpose of this document is to provide guidelies to prepare a graph model from relational data. The kind of graph model that you want to create is a key activity in this step. Please see the section 3.2 for guidelines related to performing this step.

8. Communication Plan: Identify the stakeholders for your use case and develop a communication plan to get validation and feedback on your modeling results. Key performance indicators (KPIs) for the use case

and how they relate to the model metrics should also be documented. Document your communication plan in a knowledge management system. This plan should include the frequency and format of communication with stakeholders.

A resource that is old but still relevant for a detailed discussion of the above steps is [12]. The book provides a good overview of data preparation tasks.

3.2 Graph Mapping

When you sit down to develop a modeling plan, you need to decide on the type of graph structure that is suitable for your use case. The choice of graph structure depends on the type of analysis you want to perform. This section discusses some common graph structures and their suitability for different types of analysis.

[6] and [7] provide a comprehensive discussion of some of the most common applications of graphs in enterprise applications. There are three common analysis themes relating to graph structure that we frequently encounter in enterprise applications. The first theme corresponds to a *homogeneous graph*. In such a graph, the nodes are the same type of entity. In these graphs there is a *pivotal entity* that is the focus of the analysis. For example, in a financial use case, the pivotal entity may be a *customer*. In a health care use case, the pivotal entity may be a *patient*.The analysis task typically involves predicting a property or an edge connecting these entities in the supervised learning scenario. Clustering of these entities is a common unsupervised learning scenario. If your use case is centered around a pivotal entity like a user, customer or patient, then this is a model you should consider.

The second frequently encountered theme in graph applications is the analysis of a *binary relationship*. In these applications the focus is on how two entities interact. THe analysis task is typically to extract patterns of such interactions. For example, in a retail use case, we may be interested in understanding the relationship between *customer* and *product*. In a health care use case, we may be interested in understanding the relationship between *patient* and *treatment*. In a manufacturing use case, we may be interested in understanding the relationship between *machine* and *operator*. In such cases, we may want to use a graph structure that is centered around the two entities of interest. This type of graph is a *bipartite* graph. See [10] for a detailed discussion of theory and applications of bipartite graphs. Personalization is also common motivation for this type of analysis. Summarizing the entity interactions so that you can develop personalization features is a common application task. *Machine learning for Personalization* is an emerging field, see [11] for a detailed discussion of applications and techniques. If your use case is centered around understanding the interaction between two entities, or developing personalization features, then this is a model you should consider.

Finally, the third common theme we counter is a small set of connected entities and relationships. The analysis tasks in this case are typically under-

standing the behavior of or trying to predict:

1. The property of an specific entity (*node classification*)
2. The property of a relationship between two entities (*link prediction*)
3. The property of a collection of entities (*graph classification*)

For these problems, we can map the source entities and relationships to a graph structure that replicates the entity relationships in the source system. See [17] for the modeling approach and examples. In fact, learning on graphs with this setting is the focus of *Statistical Relational Learning*. [8] provides a detailed introduction to this area. The graphs corresponding to this analysis theme have different node types and an edge connects different node types. These type of graphs are called *heterogeneous graphs.Graph Neural Networks*, the use of neural networks to solve machine learning problems on graphs is another popular solution approach for the problems discussed in this sections, see [9] and [19] for details.

The analysis themes discussed above are by no means exhaustive. These are just the common themes that we have observed in enterprise use cases. Graphs have been used in many other ways to solve a variety of problems in engineering and science. For a discussion of other applications, for example to science and engineering, please see [4]. In many niche applications, such as fraud detection or de-duplication, the graph structure is evident from the problem statement itself. This document focuses on the common themes that we have observed in enterprise use cases.

3.3 Knowledge Graph Construction

A review of the document so far will show that there are several steps in the production of a graph from relational data where modeler has to make decisions and choices. The modeler has to understand the use case, the data, the sources of heterogeneity in the data and the type of analysis that is required for the use case. The modeler also has to understand the different types of graph structures and their suitability for different types of analysis.

Knowledge graphs make it possible to capture the decisions and choices made by the modeler in a structured way. *Knowledge Management for Data Science (KMDS)* is a framework for capturing the decisions and choices made by the modeler in a structured way. The framework is implemented in Python and is available as an open source project on GitHub [13]. This framework needs work to integrate it with LLM tools.

However, the integration of KMDS with the steps outlined in this document is a work in progress. KMDS is mentioned here to highlight the importance of capturing the decisions and choices made by the modeler in a structured way.

3.4 Modeling Result Review

3.5 Modeling Result Communication

3.6 Modeling Refinement Planning

4 Examples of Implementation

4.1 Homogeneous Graph Construction for Feature Engineering

Graphs can be used to develop features for a machine learning model. In this example, we consider the problem of developing a representation for good borrowers in the SBA 7a loans dataset. This representation can be used for tasks such as developing a taxonomy of good borrowers. Understanding the heterogeneity of good borrowers can inform the design of new loan products. It can also help identify good borrowers that are similar to bad borrowers. This can help identify potential risks in the loan portfolio. In this example, we represent good borrowers as a graph and then develop a representation for this graph using a spectral embedding technique. The representation can then be used in a machine learning model. Please see this document for details.

4.2 Homogeneous Graph Construction for Representation Learning

Graphs can be used to develop a representation for a set of entities. In this example, we consider the problem of developing a representation for good borrowers in the SBA 7a loans dataset. This representation can be used for tasks such as developing a taxonomy of good borrowers. Understanding the heterogeneity of good borrowers can inform the design of new loan products. It can also help identify good borrowers that are similar to bad borrowers. This can help identify potential risks in the loan portfolio. In this example, we represent good borrowers as a graph and then develop a representation for this graph using a spectral embedding technique. The representation can then be used in a machine learning model. Please see this document for details. Note that this example is different from the previous example in that the representation is used for unsupervised learning. There is a single entity of interest, the good borrower. However, a similar approach can be used when multiple entities of interest are present and collaborate to accomplish a task. We will explore this in subsequent examples.

4.3 Bipartite Graph Construction from Relational Data

Bipartite graphs can be used to represent relationships between two types of entities. The page shows an example of using descriptive analytics to understand important characteristics of shopping behavior of customers in a retail setting. One part of the analysis examines the shopping behavior of customers in Sao

Paulo, Brazil. A bipartite graph is constructed where the bipartite vertex sets ar the cities in Sau Paulo and the products purchased by customers in these cities. The edges represent the purchase of a product by a customer in a city. Please see this document for of the analysis. The details of the bipartite graph construction and the subsequent analysis are provided in this document.

4.4 Heterogeneous Graph Construction from Relational Data

The third common theme we counter is a small set of connected entities and relationships. As part of the input to the analysis, we have the entities and the relationships between the entities. We treat these entities and relationships as a heterogeneous graph. The analysis tasks in this case are typically are:

1. Predict the property of an specific entity, *node classification*
2. Predict the relationship between two entities, *link prediction*
3. Predict the label associated with a new set of related entities, *graph classification*

@inproceedingsschlichtkrull2018modeling, @bookgetoor2007introduction, A complete example of heterogeneous graph construction and analysis for node classification is provided in this article[16]. For guidelines and theoretical background for heterogeneous graph construction from relational data, please see [17]. For a comprehensive introduction to learning graphs from relational data, please see [8].

References

[1] Leo Breiman et al. *Classification and regression trees*. Chapman and Hall/CRC, 2017.

[2] Peter Pin-Shan Chen. "The entity-relationship model—toward a unified view of data". In: *ACM transactions on database systems (TODS)* 1.1 (1976), pp. 9–36.

[3] Edgar F Codd. "A relational model of data for large shared data banks". In: *Communications of the ACM* 13.6 (1970), pp. 377–387.

[4] Diane J Cook and Lawrence B Holder. *Mining graph data*. John Wiley & Sons, 2006.

[5] Ramez Elmasri and Shamkant B Navathe. *Fundamentals of database systems seventh edition*. pearson, 2016.

[6] al. Faloutsos Christos et. *User Behavior Modeling Part*. Aug. 3, 2025. URL: https://www.youtube.com/watch?v=fMHZt4UStUg&list=PL-lbroKJyNLCAM85ENZ8g2DEiG1e5TQEo&index=1.

[7] al. Faloutsos Christos et. *User Behavior Modeling Part*. Aug. 3, 2025. URL: https://www.youtube.com/watch?v=eI-hmySej_w&list=PL-lbroKJyNLCAM85ENZ8g2DEiG1e5TQEo&index=2.

[8] Lise Getoor and Ben Taskar. *Introduction to statistical relational learning*. MIT press, 2007.

[9] William L Hamilton. *Graph representation learning*. Morgan & Claypool Publishers, 2020.

[10] Jérôme Kunegis. "Exploiting the structure of bipartite graphs for algebraic and spectral graph theory applications". In: *Internet Mathematics* 11.3 (2015), pp. 201–321.

[11] Julian McAuley. *Personalized machine learning*. Cambridge University Press, 2022.

[12] Dorian Pyle. *Data preparation for data mining*. morgan kaufmann, 1999.

[13] Rajiv Sambasivan. *GitHub - rajivsam/KMDS: Source for KMDS — github.com*. https://github.com/rajivsam/kmds. [Accessed 06-09-2025].

[14] Rajiv Sambasivan. *Understanding Coffee Prices*. https://rajivsam.github.io/r2ds-blog/posts/markov_analysis_coffee_prices/. Accessed: 2025-06-21. 2025.

[15] Rajiv Sambasivan and Sourish Das. "Classification and regression using augmented trees". In: *International Journal of Data Science and Analytics* 7.4 (2019), pp. 259–276.

[16] Jörg Schad, Rajiv Sambasivan, and Christopher Woodward. "Predicting help desk ticket reassignments with graph convolutional networks". In: *Machine Learning with Applications* 7 (2022), p. 100237. ISSN: 2666-8270. DOI: https://doi.org/10.1016/j.mlwa.2021.100237. URL: https://www.sciencedirect.com/science/article/pii/S2666827021001195.

[17] Michael Schlichtkrull et al. "Modeling relational data with graph convolutional networks". In: *European semantic web conference*. Springer. 2018, pp. 593–607.

[18] Chin-Chia Michael Yeh et al. "Matrix profile I: all pairs similarity joins for time series: a unifying view that includes motifs, discords and shapelets". In: *2016 IEEE 16th international conference on data mining (ICDM)*. Ieee. 2016, pp. 1317–1322.

[19] Jie Zhou et al. "Graph neural networks: A review of methods and applications". In: *AI open* 1 (2020), pp. 57–81.

Appendix: Vocabulary and Definitions

The Relational Model

[3] introduced the relational model, which is the logical foundation for relational databases. In this model, data is represented as relations (tables), where

each relation consists of tuples (rows) and attributes (columns). An algebraic framework for manipulating these relations is provided by relational algebra, which includes binary operations (operate on two relations) such as join and union, as well as unary operations like selection and projection (operate on a single relation). The relational model provides a mathematical framework for representing and querying data.

The Entity-Relationship Model

The entity-relationship (ER) model, introduced by [2] is a framework for describing the *semantic* data abstractions and their relationship with each other. For example, if your use case is about how students in a university register for courses, then *students, course, registration* might be a set of data abstractions. The *registration* may connect a student with a course for a particular semester. From a *data representation* point of view, all semantic abstractions may be represented as relations.

The Entity-Relationship to Relational Schema Mapping

There exist guidelines and a body of knowledge around mapping an *Entity-Relationship Model* to a *Relational Schema*. See [5], for example. The working context for this document that you have a set of extracts from a relational database, in other words, this step has been done.

Use Case Data

Your *Use Case Data* is what you want to convert to a graph. The data for your use case is a set of extracts that correspond to the entities and their relationships. As a modeler, you are expected to know:

- The semantic abstractions relevant to your use case. These are the *entities* in your data extract. You should spend sometime to develop a clear understanding of the entities in your use case.
- The *semantic relationships* in your usecase. You should understand how *entities* in your usecase are related. You should spend sometime to understand the variations that are relevant from the perspective of your use case.
- The *semantic to extract mapping* for your usecase. You should know which extract file correspond to a particular entity or relationship. The extracts are what you get from the relational databases supplying the data for your use case. As an extension, we can look at connectors that fetch data from source relational systems, but for now we will assume you have file extracts of relations and entities. Extract to semantic mapping may be a multi-step process. For example, you may need to join two or more extracts to get the data for a particular entity. You should spend sometime to understand

how the extracts map to the entities and relationships in your use case. Also you may need to transform the data in the extracts to get the data for a particular entity or relationship. You should spend sometime to understand how the extracts map to the entities and relationships in your use case.

Model Operationalization

When analysis is complete, we have a plan for how data is going to be processed and fed as input to models. Interpretation and presentation of model outputs is also determined in the analysis step. Operationalizing the model and monitoring the model is the next step. This is typically done by an MLOps team. This is outside the scope of this document.